

職務経歴書

氏名 中村 匠（なかむら たくみ）
居住地 埼玉県
E-mail takumi.nakamura.by@gmail.com
GitHub github.com/byson2562（個人） / github.com/t-nakamura2562（業務）
Portfolio byson2562.github.io
Qiita qiita.com/bython

職務要約

フューチャー株式会社に正社員として在籍し、インフラ、SRE、アプリケーション開発を横断するエンジニアとして約9年半キャリアを歩んできました。

新卒入社後は技術部隊のインフラチームに所属し、公共・ヘルスケア・保険・銀行など4つのプロジェクトでインフラの設計・構築を担当。

2019年9月よりコードキャンプ株式会社へ在籍出向し、SRE / インフラ領域のテックリードとして、アーキテクチャ改善・信頼性向上・コスト最適化・開発者体験向上などを推進。あわせて Ruby on Rails / React を用いたアプリケーション開発も担当し、インフラとアプリの両軸からプロダクト価値の最大化に貢献しています。

また、直近は生成AI活用プロジェクトのPLも兼務し、社内の生成AI環境整備やAIによる業務効率化を計画・推進しています。

スキルサマリ

SRE / インフラ

- インフラ構築・運用：パブリッククラウド（AWSなど）を用いたインフラアーキテクチャ（EC2、ECS Fargate、RDS、VPC、ALBなど）の設計・構築・運用改善を担当できます。
- IaC：CloudFormation、Ansibleを用いたコード化によるインフラ構築・運用に対応可能です。
- 監視・セキュリティ：Mackerel、AWS WAF、Security Hubなどを用いた監視・セキュリティ対策の設計・実装・運用を経験してきました。
- CI/CD：CircleCI、GitHub Actionsを用いたCI/CDパイプラインによるテスト・ビルド・デプロイの設計・実装・運用を主担当として実施できます。
- 信頼性向上：E2Eテストやユニットテストの実装やCIによる自動化に加え、Dependabotによるライブラリの脆弱性管理、rubocopによるコード品質管理の実装・運用経験があります。

アプリ

- フロントエンド：Reactを用いた複数コンポーネントにわたる機能開発に対応できます。
- バックエンド：Ruby on Railsを用いた機能の設計・開発が可能で、PdMと協働して要件定義から携わるケースも経験しています。

職歴

フューチャー株式会社

- **在籍期間**：2016年9月～現在（約9年半）
- **雇用形態**：正社員
- **事業内容**：ITコンサルティング・受託開発
- **従業員数**：連結：3,624名（2025年12月末現在）。
- **概要**：入社後は各PJで、インフラレイヤの設計・構築を担当。2019年9月よりコードキャンプ株式会社へ在籍出向。

出向先：コードキャンプ株式会社（2019年9月～現在）

- **担当職種**：SRE / インフラテックリード / アプリケーションエンジニア
- **事業概要**：法人向けDX教育の提供およびオンラインプログラミングスクール「CodeCamp」の運営。
- **サービス規模**：累計受講者数 5万人以上・累計提供法人数 300社以上 / ECS 上で稼働する主要サービス 4つを含む本番基盤を運用。

SRE / インフラ

【1】継続的な基盤整備・運用

担当期間：2019年9月～現在 役割：SRE / インフラテックリード（2021～現在）、インフラ主担当（2019～2020）

背景：

- 参画時、社内にはアプリケーションエンジニア数名のみが在籍しており、SRE / インフラ領域の専任エンジニアは不在で、インフラ環境も未整備な状態だった。社内で唯一 SRE / インフラ領域に知見のあるエンジニアとして、本番環境の信頼性を担保し、開発者がプロダクトに専念できる環境を整備。
- 現在においてもテックリードとして継続的に課題発見・解決を行っている。

取り組み：

- 監視
 - Zabbixは存在していたがほぼ使われていなかったため、シンプルかつUIフレンドリーでアプリエンジニアでも扱いやすいMackerelへ移行し、外形監視などアラートの実装と、実際に使える監視体制を構築。障害アラート発生時の一次対応も主担当として実施。
 - Mackerelのダッシュボードやアラート設定についてはTerraformによって設定をコード化。
 - ログの棚卸しと logrotate 整備により、ストレージの適正化と障害時のログ調査コストを削減。
- セキュリティ
 - インフラレイヤのセキュリティリスクが可視化されていなかったため、Security Hub を導入しセキュリティ状態を可視化。
 - DoS攻撃など悪意のあるリクエストへの防衛手段が存在しなかったため、AWS WAFを導入しルールを設定。
- インフラ構築・改善
 - データエンジニアの参画に伴い、MySQL → BigQuery の ETL 基盤（Embulk + digdag）を構築し、データ分析基盤を整備。
 - cronや運用スクリプトが各サーバに散らばって配置されており、どこで何が動いているのかを把握できていない状態であったため、Rundeck に集約。
 - ユーザー数の増加に伴い、SMTPサーバーをGmailからSendGrid移行し、配送の信頼性を向上。
 - OS/MWのEOL管理およびアップデート対応。
- コスト管理
 - AWSのコスト最適化（過剰なリソースの精査・savings plansの適正化により月あたりのAWSコストを約20%削減）と定期的な監視。
- トイル削減
 - Dependabotによるライブラリの脆弱性可視化と継続的なアップデートの自動化。
 - ステージング/検証環境の起動・停止をRundeckジョブでスケジューリング。

成果：

- 可観測性・セキュリティ・自動化を一通り整備し、後の本格的なアーキテクチャ刷新（ECS Fargate 移行・CI/CD 再設計）の下地を作った。

Mackerel

AWS WAF

AWS Security Hub

CloudFormation

Ansible

Rundeck

BigQuery

Embulk

digdag

Docker

Dependabot

【2】RSpec (System Spec) を用いた E2E テストの実装

担当期間：2023年後半 役割：テックリード（テスト設計・実装・運用整備）

背景・課題：

- 広範に影響のあるリリース（大規模な機能開発やRailsのアップデートなど）後に障害が発生するケースが年に3～4回程度発生しており、リリース前の品質担保の仕組みが必要な状態だった。
- 上記における対策が、PdMによるリグレッションテストとなっていたが、ケースの網羅性やテストの自動化が不十分で、自動的かつ継続的な品質チェックの仕組みとしては機能していなかった。

取り組み：

- E2Eテストを導入することで、継続的かつ自動的にリリース前の品質担保がされる状態を目指すことを起案し、テックリードとして設計・実装・運用整備を一貫して実施。
- PdMを巻き込んでケース別の障害レベルの設定を行い、クリティカルなものをE2Eテスト対象として選定。
- ツールとしては、RSpec と統合されており Rails アプリの E2E テストでメジャーな System Spec を採用。
 - 技術選定観点をまとめた記事：<https://qiita.com/bython/items/4d79ade2e8da5378e8b6>
- テストの実装およびCI パイプラインへの組み込みによるテストの自動化を行い、E2Eテストをpassした時のみ master ブランチへのマージが許可されるように設定。リリースにおいてE2Eテストによる品質が担保されている状態を実現。
- 他メンバーにはプルリクを元に口頭ベースでナレッジを共有。

成果：

- 主要な導線を E2E で網羅し、年に3～4回起きていたリリース後の機能障害を年0～1回程度に抑制。プロダクトの信頼性向上に貢献。
- 現在においても運用が継続されており、他メンバーによるケース追加も継続的に行われている。

RSpec (System Spec)

GitHub Actions

Ruby on Rails

【3】EC2 中心の構成を ECS Fargate へ移行・運用基盤の整備

担当期間：2023年～2024年 役割：テックリード（技術選定・設計・実装・運用定着を一貫担当）

背景・課題：

- EC2ベースの構成でOSパッチ適用などの保守運用が継続的に発生しており、また、インプレースデプロイのビルド負荷によるリリースタイミングの調整が発生する点や単一サーバ運用による単一障害点も課題だった。
- 会社の事業としてtoB向けに事業をピボットしていく中で、障害の影響度も高まっており、信頼性向上のためのアーキテクチャ刷新が必要な状態だった。
- また、本番リリース前の検証環境がステージング環境しか存在せず、リリースタイミングが重なる際には検証環境の利用調整が発生していた。

取り組み：

- まず、AmazonLinuxのEOLによるリプレースのタイミングで、ブラックボックスとなっていたサーバの設定を棚卸しし、Ansibleによって見える化。
- 次に、ローカル環境にDockerを導入し、メンバー参画時における開発環境構築工数（1～2日）をほぼゼロに短縮しつつ、コンテナでアプリケーションが動作することを担保。
- 最後に、ECS Fargate 移行の設計・実装およびデプロイフローの再設計・実装を行い、本番環境へ切り替え。
- 合わせて、本番・ステージングと同等構成の ECS 環境を開発者ごとに立ち上げられる仕組みを構築し、エンジニアごとに専用の検証環境を提供できる状態に改善。

成果：

- サーバーレス構成になったことにより、OS パッチ運用工数を不要化。
- 冗長構成により単一障害点を解消し、サービスの信頼性を向上。
- インプレースデプロイのビルド負荷も解消され、リリースのタイミング調整も不要化。
- 開発者がECS環境を自律的に立ち上げられるようになり、プロダクト側への実装確認を行いやすい状態に改善。

ECS Fargate

EC2

ALB

CloudWatch

CloudFormation

GitHub Actions

Docker

【4】CI/CD パイプラインの再設計

担当期間：2024年後半 役割：テックリード（CI/CD設計・移行推進）

背景・課題：

- GitHub Actions の機能成熟により、CircleCI と GitHub Actions の機能差が小さくなってきており、また市場でも GitHub Actions の採用が広がっていた。
- バージョン管理として使っている GitHub に CI ツールを統合することで、設定の一元化と運用負荷の削減を狙った。
- また、パイプラインが適正化されておらず、CIを無駄に実行したりキャッシュが有効活用されていないケースがあった。（fixtureの変更でassetをビルドしてしまうなど）

取り組み：

- 既存 CircleCI ジョブの棚卸しと可否を判定した上でGitHub Actions へ移行。
- パイプラインを適正化し、変更内容によって適切なステップ実行やキャッシュを利用するように修正。

成果：

- ツールを GitHub に一元化し、開発フローとの統合性を向上。
- キャッシュ活用により CI 実行時間を最大約5分短縮。

GitHub Actions

CircleCI

Docker

アプリケーション開発

【1】各種機能開発

担当期間：2019年～現在 役割：機能開発主担当

背景・課題：

- 機能実装を通じて、ユーザーの学習体験や社内メンバーの業務効率化を目指している。
- フロントエンドはReact、サーバーサイドはRailsを採用。

取り組み（代表的なもの）：

- wicked_pdf/Prawnによる帳票機能開発
- アセスメント機能（簡易的なGoogleフォームのようなイメージ）開発
- インボイス制度対応
- 社内業務管理画面の機能改修

Ruby on Rails

React

【2】生成 AI を活用したソースコード添削機能の開発

担当期間：2025年前半 役割：機能開発リード（要件定義・設計・実装・運用整備）

背景・課題： プログラミング学習者のコード提出に対するフィードバックを高品質・高速化することで、学習体験の向上を目指した。

取り組み：

- 起票者が業務委託の教材開発者だったため、要件整理から巻き取り、技術選定（LLM API の選定）までリード。LLM API としては、API 経由での利用がメジャーな OpenAI を選択。
- React + Railsで機能実装しリリース。

成果：

- 主要な学習機能の一つとして本番運用継続中。コードフィードバックの品質・応答速度を向上させ、ユーザー体験の向上に貢献。

Ruby on Rails

React

OpenAI API

【3】学習環境基盤の刷新（API の Rails 移行 + 開発環境の code-server 化）

担当期間：2025年10月～2026年5月（うち2025年11～12月は育児休業） 役割：PM兼テックリード（技術選定・設計・実装）

背景・課題：

- 学習者向け開発環境（オンラインエディタ）として AWS Cloud9 を採用していたが、AWS による新規アカウントでの利用制限が発表され、サービスの継続性に不安がある状態となった。
- 企業研修の提供において、オンラインエディタに依存している状態のため、代替手段の検討が必要となった。
- 加えて、学習環境を払い出す API が過去の担当者による Laravel 実装のまま残っており、社内標準の Rails と乖離して保守性に課題があった。
- これらの課題を解決するため、API の Rails への移行と、オンラインエディタの代替手段の検討・導入をリードすることとなった。

取り組み：

- PJ全体のリード
- 代替エディタとして複数候補を比較し、self-hosted 運用可能な code-server を採用することに決定。
- 約27,800ステップの Laravel から Rails への移行設計・実装。
- Cloud9 から code-server への移行に際しては、特定の企業でまずcode-server を利用してもらい、フィードバックをもらいながら段階的に切り替える方式を採用し、移行リスクを最小化。
- 上記実現のための社内関係者との調整・コミュニケーション。

成果：

- 社内標準技術への統一により API 保守の属人化を解消し、保守性を改善。
- Cloud9 廃止リスクを回避し、学習者向け開発環境のサービス継続性を確保することで、事業の継続性を担保。

Laravel

Ruby on Rails

Docker

code-server

AWS SDK

CloudFormation

GitHub Actions

フューチャー株式会社（2016年9月～2019年9月）

- 担当職種：** インフラエンジニア

- **案件概要：** 公共・ヘルスケア・保険・銀行など幅広い業界のPJにおいて、インフラ基盤の設計・構築・運用改善を担当。

【1】茨城県 / セキュリティクラウド構築（公共）

担当期間：2016年12月～2017年4月 規模：Linux・Windowsサーバ約10台 チーム：3名（職位：メンバー）

取り組み：

- ウイルス検知・ログ収集/分析サーバ（Linux）の結合試験・総合試験・リリース・運用
- 運用手順書の作成・運用スクリプト開発

CentOS Windows Server Elasticsearch Logstash Kibana

【2】三菱商事株式会社 / ヘルスケアプラットフォーム構築

担当期間：2017年4月～2017年9月 規模：WebAPサーバ12～16台（AutoScaling）／DBサーバ6台 アプリチーム：10名 / インフラチーム：3名（職位：メンバー）

取り組み：

- Ruby on Railsによるアプリケーション改修・追加機能開発、結合試験
- AWS基盤の運用監視（CloudWatch）・基盤移行スクリプト開発
- 運用手順書の作成

AWS Amazon Linux Ruby on Rails RSpec PostgreSQL (RDS) CloudFormation Ansible CloudWatch
Jenkins

【3】SOMPOフランティ株式会社 / 新規保証事業システム構築

担当期間：2017年9月～2017年12月 規模：WebAPサーバ12～16台 / バッチ4台 / 分析3台 / DBサーバ9台 チーム：2名（職位：メンバー）

取り組み：

- AWS基盤の詳細設計・構築（VPC、EC2、RDS等）
- DevOps環境構築（Jenkins / GitLab / CloudFormation / Ansible）
- Linuxサーバ構築

AWS Amazon Linux CloudFormation Ansible Java Oracle RDS Jenkins

【4】千葉銀行株式会社 / 新融資支援システム構築（金融）

担当期間：2017年12月～2019年9月 規模：WebAP・バッチ・DB・OCR・FAX等 計約30台規模のオンプレ基盤 チーム：2名（職位：メンバー）

取り組み：

- 基盤の要件定義～総合試験まで担当（Windows/Linuxサーバ構築、MW/SWセットアップ、CI環境整備）
- 監視・運用ジョブ・ログ改廃の設計および実装（JP1 AJS / PFM、PowerShell）

Windows Server Linux Java PowerShell DB2 / Oracle / PostgreSQL JP1 (AJS / PFM) Jenkins / GitLab

生成 AI 全社活用プロジェクト（PL）

期間：2026年2月～現在 役割：プロジェクトリード（PL）

背景・概要：

- 生成AIを活用した業務効率化を全社規模で推進するため、技術選定から社内展開・定着まで横断的に担うPLを担当。

取り組み：

- PJ計画の策定と関係者調整
- ツール選定（n8n・Difyを採用）と検証環境の整備
- 業務自動化ワークフローの設計・モック実装（n8n / Dify）
- NotebookLMを活用した社内ナレッジ検索機能の構築
- 全社的なAI利用環境の設計・構築

中間成果：

- ツール選定～検証環境構築～業務自動化ワークフローのモック構築までを約3ヶ月で完了。
- 現在はBiz側と共同で業務効率化ワークフローのブラッシュアップと、全社的なAI利用環境の設計・構築を進行中。

n8n

Dify

NotebookLM

OpenAI API

Claude API

AWS

クラウド・AWS 教材の執筆および Linux 教材のレビュー

期間：2025年中旬 役割：執筆・レビュー担当

背景・概要：

- 法人研修で使用するクラウド・AWS カリキュラムの設計および教材の執筆と、Linux 教材のレビューを担当。
- AWS認定資格保有者（AWSSAA・現在は失効済み）として、AWSの基礎から実践までをカバーする内容を執筆し、理解度テストも作成。Linux教材については、内容の正確性やわかりやすさの観点でレビューを実施。

取り組み：

- 法人研修向けのカリキュラム設計および教材の執筆：クラウド基礎（3章）・AWS基礎（7章）・AWS実践（8章）、計18章
- 理解度テストの作成：AWS認定資格をベースとした理解度テストの設計・作成
- Linux基礎教材（6章）のレビュー

成果：

- 法人研修コンテンツのデリバリーに貢献。
- 内容については一定の評価を得ることができ、翌年のリピート受注につながった。

AWS

Linux

教材執筆

スキルセット

カテゴリ	技術・ツール	実務経験年数
クラウド	AWS（ECS / EC2 / RDS / S3 / CloudWatch / ALB / IAM / Lambda ほか）	約7年
IaC	CloudFormation、Ansible	約7年
コンテナ	Docker、ECS Fargate	約5年
バックエンド	Ruby on Rails	約7年
フロントエンド	React、JavaScript / TypeScript	約5年
データベース	MySQL、PostgreSQL、BigQuery	約7年
CI/CD	GitHub Actions、CircleCI	約7年
可観測性	CloudWatch、Mackerel	約7年
テスト	RSpec	約3年
LLM API	OpenAI API	約1年
AI ワークフロー	n8n、Dify	約4ヶ月
OS / 基盤	Linux（Amazon Linux / Ubuntu）	約9年
バージョン管理	Git / GitHub	約7年

自己 PR

SRE / インフラ領域のテックリードとしての実行力

コードキャンプでは、SRE / インフラ領域のテックリードとして、AWSの主要サービスを活用したインフラ構築・運用に加え、信頼性向上・コスト最適化・開発者体験向上といったSRE観点でのプロダクト基盤の技術選定・設計・改善を続けてきました。これらを通じて、技術選定・設計・実装・運用改善まで一貫してリードし、プロダクト基盤の価値最大化にコミットする実行力を発揮してきました。

フルスタックな技術領域

SRE / インフラ領域のテックリードでありつつ、React + Railsによる機能開発など、アプリケーション開発もフロントからバックエンドまで自ら実装して推進してきました。インフラとアプリの両軸を一人称で推進できるため、技術選定や設計において、両者の観点を踏まえた最適解を追求できます。

エンジニアリングを目的化せず、事業・プロダクトの価値最大化を追求

自身が担当する実装領域においては、与えられた要件をただ実装するのではなく、プロダクトや組織にとって何が本当に必要かを考えながら、仕様の提案を含めてアウトプットしています。また、学習環境基盤の刷新や生成AI活用プロジェクトなど、案件によっては自らディレクションやプロジェクトマネジメントも担いながら、事業・プロダクトの価値最大化にコミットしています。

今後の志向

SRE / インフラ領域を軸としつつ、レイヤを問わず会社・組織・プロダクトの価値が最大化されるよう貢献していきたいです。

特に SRE 領域においては、これまで監視・アラートの整備や E2E テストによる品質担保など、信頼性の土台を作る取り組みを進めてきました。今後はその延長として、SLI / SLO の策定やエラーバジェットの運用など、信頼性を定量的に合意形成し、リリース判断やプロダクト戦略と接続する SRE プラクティスに踏み込んでいきたいと考えています。

以上